

Diseño de un Lenguaje de Programación Transpilado de Esperanto a Rust: Integración de Ergonomía C++ y Desarrollo Autónomo mediante Claude Code

Introducción a la Arquitectura de Lenguajes Localizados

La hegemonía del idioma inglés en la sintaxis, las palabras clave y la infraestructura de los lenguajes de programación ha creado una barrera cognitiva histórica para los desarrolladores no angloparlantes en todo el mundo. Aunque han existido esfuerzos pioneros para desarrollar lenguajes en otros idiomas, como *Rouille* (una traducción de Rust al francés), *Wenyan* (basado en chino clásico) o *Linotte*, la inmensa mayoría de estas iniciativas sacrifican el rendimiento industrial a cambio de la accesibilidad educativa. Sin embargo, el objetivo de construir un lenguaje de programación moderno que ofrezca las garantías irrestrictas de seguridad de memoria de Rust, incorpore la flexibilidad ergonómica y el paradigma orientado a objetos de C++, y cuya base léxica esté construida estrictamente en Esperanto, representa un avance sin precedentes en la ingeniería de compiladores. Este proyecto, de naturaleza enteramente de código abierto, requiere una meticulosa orquestación técnica, desde el análisis morfológico de las palabras clave hasta la implementación de una cadena de herramientas de compilación que opere de manera fluida y nativa.

El Esperanto, creado por L. L. Zamenhof a finales del siglo XIX, es un idioma planificado que exhibe una regularidad morfológica y sintáctica absoluta, desprovista de las excepciones irregulares que plagan a los lenguajes naturales como el inglés. Esta consistencia matemática lo convierte en un candidato excepcional como base para un analizador léxico (lexer) y un analizador sintáctico (parser). Las reglas del Esperanto dictaminan terminaciones fijas para las categorías gramaticales: todos los sustantivos terminan invariablemente en -o, los adjetivos en -a, los adverbios derivados en -e y los verbos en infinitivo en -i. Durante la fase de tokenización del código fuente, esta predictibilidad permite al analizador clasificar semánticamente los identificadores con una eficiencia algorítmica superior, reduciendo la ambigüedad inherente en el diseño de compiladores y facilitando la construcción de Árboles de Sintaxis Abstracta (AST) de alta fidelidad.

Para lograr este cometido de manera efectiva, el diseño no consistirá en la escritura de un compilador nativo desde cero, sino en el desarrollo de un transpilador avanzado (un compilador de fuente a fuente). El ecosistema del compilador oficial de Rust (rustc) ejecuta una serie de transformaciones sumamente complejas: traduce el texto crudo a tokens, luego a un AST, posteriormente a representaciones intermedias de alto nivel (HIR), nivel tipado (THIR) y nivel medio (MIR), para finalmente alimentar a LLVM-IR (Low Level Virtual Machine Intermediate Representation) y generar código máquina. Reimplementar el comprobador de préstamos

(borrow checker) y el análisis de tiempos de vida (lifetimes) a nivel del MIR requeriría décadas de ingeniería. Por consiguiente, la estrategia óptima es construir un frontend que realice el análisis léxico y sintáctico del código en Esperanto, aplique las transformaciones ergonómicas heredadas de C++, y emita código Rust válido e idiomático. Herramientas de transpilación análogas, como c2rust, han demostrado que la emisión de código Rust a partir de un AST extranjero es un método robusto para heredar automáticamente la seguridad de memoria, el polimorfismo y el ecosistema de paquetes nativo (crates.io) sin sacrificar el rendimiento de ejecución.

La magnitud de este proyecto requiere un enfoque de ingeniería moderno. La totalidad de la base de código, la documentación y las pruebas unitarias se desarrollarán utilizando la asistencia autónoma de *Claude Code*, un agente de inteligencia artificial integrado directamente en la terminal que posee la capacidad de editar archivos locales, ejecutar comandos del sistema, revisar infraestructuras complejas y mantener la integridad del repositorio mediante el uso de herramientas de línea de comandos.

Diseño Morfológico y Léxico del Lenguaje en Esperanto

La adaptación de la sintaxis de Rust al Esperanto demanda una investigación exhaustiva de las convenciones terminológicas de ambos ecosistemas. El vocabulario informático en Esperanto ha sido normalizado y estandarizado mediante proyectos colaborativos y diccionarios técnicos como *Komputeko* (Prikomputila terminokolekto), el *Plena Ilustrita Vortaro de Esperanto* (PIV), y el *Komputada Leksikono*. Estos repositorios proporcionan equivalencias exactas para la jerga técnica, evitando la creación de neologismos innecesarios y garantizando que el lenguaje resultante sea comprensible para cualquier hablante de Esperanto en el mundo.

El compilador original de Rust clasifica su sintaxis en palabras clave estrictas, reservadas y débiles. Las palabras clave estrictas, como `fn`, `let`, `mut`, y `struct`, definen las declaraciones fundamentales y el control de flujo del lenguaje, y no pueden utilizarse como nombres de variables o identificadores. La traducción de estas palabras clave al Esperanto se ha diseñado meticulosamente, priorizando raíces etimológicas internacionales y aprovechando el sistema de afijos aglutinantes del idioma para denotar las mecánicas de seguridad de memoria.

Palabra Clave en Rust	Función Semántica / Estructural	Traducción al Esperanto	Justificación Etimológica y Morfológica
fn	Definición de funciones	fk o funk	Derivado de la raíz <i>funkcio</i> (función). Mantiene la brevedad ergonómica característica de Rust.
let	Declaración y enlace de variables	estu	Empleo del verbo <i>esti</i> (ser/estar) en modo volitivo o imperativo (terminación -u), indicando "que sea".
mut	Denotación de mutabilidad en	ŝanĝ	Raíz del verbo <i>ŝanĝi</i> (cambiar). Su uso

Palabra Clave en Rust	Función Semántica / Estructural	Traducción al Esperanto	Justificación Etimológica y Morfológica
	referencias y variables		evoca la propiedad intrínseca de ser modificable (<i>ŝanĝebla</i>).
struct	Definición de estructuras de datos	strk	Abreviación del sustantivo <i>strukturo</i> , alineándose con la nomenclatura técnica de C++ y Rust.
impl	Implementación de métodos o rasgos inherentes	realigu	Verbo <i>realigi</i> (hacer real, implementar) en modo volitivo, indicando la materialización de un comportamiento.
trait	Definición de interfaces y comportamientos compartidos	trajto	Traducción directa del inglés; el sustantivo <i>trajto</i> se utiliza habitualmente para denotar características o atributos.
enum	Definición de tipos algebraicos enumerados	enumb	Abreviación estandarizada del término <i>enumbro</i> o <i>enumeracio</i> .
match	Bifurcación por coincidencia de patrones (pattern matching)	kongruu	Verbo <i>kongrui</i> (coincidir, encajar) en modo volitivo. Refleja la evaluación exhaustiva de patrones.
loop	Declaración de un bucle infinito incondicional	ripetu	Verbo <i>ripeti</i> en modo volitivo. Ordena al procesador repetir la operación de manera indefinida.
for	Iteración sobre colecciones y rangos	por	Preposición fundamental <i>por</i> (para/por), utilizada para establecer el propósito o el límite de la iteración.
in	Inclusión sintáctica dentro del bucle iterativo	en	Preposición fundamental <i>en</i> (en/dentro de), indicando pertenencia

Palabra Clave en Rust	Función Semántica / Estructural	Traducción al Esperanto	Justificación Etimológica y Morfológica
			a la colección evaluada.
while	Bucle condicionado a una expresión booleana	dum	Conjunción temporal <i>dum</i> (mientras), que evalúa la continuidad del ciclo.
if / else	Control de flujo mediante bifurcación condicional	se / alie	Conjunciones <i>se</i> (condicional "si") y <i>alie</i> (adverbio "de otro modo" o "si no").
return	Devolución anticipada del flujo de ejecución	redonu	Uso del prefijo de repetición <i>re-</i> sobre el verbo <i>doni</i> (dar). Significa "devolver".
pub	Denotación de visibilidad pública en el sistema de módulos	pub	Abreviación natural del adjetivo <i>publika</i> , manteniendo compatibilidad visual y brevedad.
use	Importación de espacios de nombres y resolución de rutas	uzu	Modo imperativo del verbo <i>uzi</i> (utilizar), que introduce símbolos al alcance actual.
crate	Raíz jerárquica del sistema de empaquetamiento y enlazado	kesto	Traducción literal de <i>crate</i> (cajón o caja de embalaje), respetando la metáfora de Cargo en el ecosistema Rust.
unsafe	Desactivación explícita de las garantías del comprobador de memoria	malsekura	Uso magistral del prefijo <i>mal-</i> (antonimia directa) sobre el adjetivo <i>sekura</i> (seguro), creando "inseguro" de forma perfectamente regular.

Adaptación de los Tipos Primitivos y Colecciones

En la especificación oficial de Rust, los tipos primitivos están profundamente entrelazados con la arquitectura del hardware, dictando el tamaño de los registros en la memoria, las operaciones aritméticas válidas y la gestión de punteros. Estos tipos abarcan representaciones escalares (enteros con y sin signo, coma flotante, caracteres, booleanos) y compuestos (tuplas, arreglos). Para el lenguaje en Esperanto, la transpilación debe interceptar estas declaraciones semánticas y transformarlas a la sintaxis del lenguaje de destino.

El sistema de tipos propuesto se basa en una nomenclatura lógica y deductiva, garantizando

que el usuario asimile la carga de memoria de la variable inmediatamente.

- **Tipos Enteros con Signo:** Los tipos nativos de Rust i8, i16, i32, i64, i128, y el tipo dependiente de la arquitectura isize, derivan conceptualmente del término "integer". En el diseño del nuevo lenguaje, se adopta la raíz *entjero* (entero). La nomenclatura resultante se simplifica como ent8, ent16, ent32, ent64, ent128. El tipo de tamaño variable de puntero (isize) se designa como entg (entjero granda).
- **Tipos Enteros sin Signo:** En Rust se representan como u8 hasta u128 y usize. Utilizando el concepto matemático de números naturales, el Esperanto emplea la raíz *natura*. Así, los tipos se exponen como nat8, nat16, nat32, nat64, nat128. El tamaño de puntero sin signo se traduce como natg.
- **Coma Flotante:** Los tipos f32 y f64 cumplen con el estándar IEEE-754. La traducción técnica precisa para "floating point" es *glitkomo*. Por simplicidad, se abrevian a glit32 y glit64.
- **Tipos Lógicos (Booleanos):** El tipo primitivo bool se mapea a bulea (el adjetivo booleano en Esperanto). De forma equivalente, los valores literales true y false se traducen estrictamente como vera y malvera, aprovechando nuevamente el prefijo de oposición mal-.
- **Cadenas de Texto y Caracteres Escalares:** El tipo fundamental de porción de cadena en Rust (&str) se traduce usando la raíz *ĉeno* (cadena), presentándose como &ĉen. Para el carácter escalar Unicode de cuatro bytes (char), se adopta la contracción kar (derivado de *karaktero* o *litero*).
- **Colecciones de la Biblioteca Estándar:** Para mantener el aislamiento idiomático total, el compilador interceptará colecciones comunes como vectores dinámicos (Vec<T>) y cadenas alojadas en el montículo (String), reemplazándolas por alias en Esperanto durante la etapa del Árbol de Sintaxis Abstracta. De esta forma, el usuario interactuará con Vektoro<T> y Teksto, los cuales se desugarizarán (desugaring) automáticamente a sus equivalentes exactos en Rust al momento de la transpilación.

Simbiosis Estructural: Ergonomía de C++ en el Ecosistema Rust

Un requerimiento cardinal del diseño es dotar al lenguaje de la fluidez y las "mejores características" de C++, contrarrestando la conocida curva de aprendizaje de Rust impuesta por la verbosidad de las declaraciones de tiempos de vida explícitos y la estricta composición de estructuras. Aunque Rust promueve la composición por encima de la herencia y exige un manejo exhaustivo de resultados mediante tipos enumerados como Result<T, E>, el nuevo lenguaje en Esperanto actuará como un preprocesador inteligente que inyectará azúcar sintáctico (syntactic sugar) para suavizar estas interacciones, operando a nivel del analizador léxico y sintáctico antes de que la herramienta original lo evalúe.

El diseño del transpilador incorpora resoluciones a tres de las fricciones más comunes para los desarrolladores de C++ que migran a sistemas de tipos seguros:

1. **Parámetros por Defecto y Sobrecarga de Funciones:** Rust rechaza arquitectónicamente la sobrecarga de funciones y los argumentos por defecto, obligando al uso del patrón constructor (Builder Pattern) o la definición de múltiples funciones con nombres explícitos. El lenguaje en Esperanto ofrecerá soporte nativo para funciones sobrecargadas. El transpilador analizará las firmas de las funciones definidas con el mismo identificador, calculará la aridad y los tipos de los argumentos, y generará código

Rust válido aplicando "mangling" de nombres (una técnica análoga a la utilizada por C++ y LLVM IR donde el nombre de la función se concatena con el hash de sus tipos) o generando automáticamente una macro declarativa de Rust en el backend que dirija (dispatch) las llamadas a la función interna correcta según los parámetros provistos.

2. **Manejo Simplificado de Errores Estilo Excepciones:** Mientras que C++ utiliza bloques try/catch para capturar excepciones en tiempo de ejecución, Rust prefiere el manejo explícito de valores a través del operador ? y el desempaqueado de resultados. El nuevo lenguaje proveerá un constructo lógico en Esperanto, como provu / kaptu (try / catch), que en el fondo el transpilador convertirá en un elegante manejo de correspondencia de patrones (pattern matching) sobre tipos Result, garantizando que ningún error no controlado cause un colapso del sistema (panic), pero permitiendo al programador una sintaxis de lectura lineal y continua.
3. **Herencia Sintáctica mediante Composición Oculta:** Para emular la herencia de clases de C++, el transpilador permitirá una declaración de clase donde un strk (struct) puede "extender" otro. Durante el descenso del Árbol Sintáctico, esta extensión se expandirá de manera invisible insertando los campos de la estructura padre directamente dentro de la estructura hija y generando automáticamente bloques impl con las delegaciones de métodos correspondientes, proveyendo herencia simulada de costo cero en un entorno puramente orientado a la composición.

Arquitectura del Transpilador: Análisis y Generación de Código

La viabilidad técnica de este lenguaje recae sobre su infraestructura de compilación. En lugar de procesar los tokens de Esperanto directamente hacia LLVM-IR, el software transcribirá el código nativo a tokens de Rust. Esto implica una arquitectura dividida en capas lógicas estrictas, sostenida por las poderosas bibliotecas de macros y análisis del propio Rust, garantizando la compatibilidad futura con cualquier evolución del compilador central.

Análisis Léxico mediante Gramáticas Formales

El primer módulo de la herramienta es el escáner léxico. Puesto que el Esperanto requiere soporte mandatorio y nativo de UTF-8 para procesar sus caracteres especiales de manera eficiente, no es posible utilizar analizadores léxicos simples basados en ASCII. El análisis léxico se desarrollará utilizando la biblioteca pest, un potente generador de analizadores en Rust basado en la Gramática de Expresiones de Análisis (Parsing Expression Grammar - PEG). El archivo de gramática PEG (por ejemplo, esperanto.pest) codificará formalmente cada regla del lenguaje. Los identificadores se definirán mediante expresiones regulares complejas que abarquen el bloque Unicode completo de caracteres latinos extendidos para el Esperanto, garantizando que variables denominadas ĉefa_funkcio o ĝusta_valor se analicen correctamente. Al mismo tiempo, el lexer aplicará la regla de la coincidencia más larga (maximal munch rule) para discernir entre palabras clave como estu y variables que incidentalmente comienzan con la misma secuencia de letras.

Análisis Sintáctico y Generación del AST Propietario

Una vez que la cadena de texto cruda es transformada en un vector de tokens seguros, la

etapa de análisis sintáctico toma control para construir el Árbol de Sintaxis Abstracta (AST). El AST no es simplemente una transcripción; es la representación estructurada y jerárquica de la lógica del programa, que captura el significado de las expresiones prescindiendo de la puntuación superficial.

El compilador definirá enumeraciones fuertemente tipadas en Rust para representar estos nodos semánticos. Por ejemplo, se modelará un nodo *Esprimo* (Expresión) que contenga variantes como *BuleaLiteral* o *DulateraOperacio*, y un nodo *Deklaro* que abarque estructuras de control y funciones. Durante este proceso, se mantendrá un riguroso seguimiento de los metadatos posicionales de cada token (*Spans*). La preservación del *Span* (línea, columna y longitud del identificador en el documento original) es el pilar fundamental que permitirá que, cuando el compilador final de Rust encuentre un error de tipos o violaciones de préstamos de memoria, el transpilador intercepte el mensaje de error y proyecte la advertencia visualmente sobre la línea exacta del código fuente original en Esperanto, brindando una experiencia de usuario ininterrumpida.

Generación de Código con 'Syn' y 'Quote'

La verdadera maravilla de este diseño arquitectónico reside en la última capa de transpilación, ejecutada casi exclusivamente mediante la interacción de las bibliotecas internas *syn* y *quote*. La biblioteca *syn* es el estándar en el desarrollo de macros procedimentales para Rust; ofrece estructuras de datos completas y exhaustivas (*syn::File*, *syn::Item*, *syn::Expr*, *syn::Type*) capaces de representar cualquier permutación de código Rust lícito.

El transpilador recorrerá el Árbol Sintáctico abstracto del Esperanto. Para cada nodo evaluado, invocará la macro *quote!*. Esta funcionalidad de cuasi-comillas interpolará las variables dinámicas extraídas del AST (como los nombres de los identificadores o los tipos de datos traducidos de *ent32* a *i32*) y emitirá directamente un flujo de tokens de Rust (*TokenStream*) de alta fidelidad. El flujo resultante no es un archivo de texto frágil y susceptible a errores tipográficos generados por el sistema, sino código semánticamente verificado a nivel de sintaxis que puede ser consumido por el ejecutable *rustc* o la herramienta *cargo* sin ninguna intermediación manual.

Metodología de Ejecución Autónoma mediante Claude Code

Llevar a la realidad la totalidad de la arquitectura de este compilador representa un esfuerzo volumétrico inmenso, particularmente porque se requiere que la programación sea ejecutada exclusivamente por *Claude Code*, un agente conversacional basado en la línea de comandos. Las operaciones a nivel de compiladores, como el diseño del analizador PEG, el recorrido transversal del AST y el manejo de los flujos de *stdout* y *stderr* de los procesos de sistema de transpilación, exigen un contexto ininterrumpido a lo largo de docenas de iteraciones. Sin embargo, agentes de terminal como *Claude Code* implementan mecanismos agresivos de gestión de memoria llamados "compactación automática del contexto" (*auto-compaction*). Cuando el intercambio de lectura y escritura de archivos acerca la sesión a la capacidad máxima de tokens (típicamente desencadenado entre el 50% y el 85% de la ventana), el agente genera un resumen de la sesión pasada y purga los registros detallados para evitar la terminación abrupta. En el desarrollo de compiladores, la compactación resulta devastadora; la pérdida de directivas específicas sobre el mapeo léxico del Esperanto o la sutil jerarquía de

nodos del analizador syn induce a "alucinaciones" de código que quiebran la compilación de la base de manera irreparable.

Por ende, el plan de acción maestro debe blindar las reglas del proyecto integrando instancias avanzadas a través de enrutadores y explotando el sistema de "Hooks" del entorno.

Intervención 1: Enrutamiento de Modelos de Larga Contextura

Para soportar las densas arquitecturas de AST, el entorno predeterminado de Claude Code se redirigirá para operar sobre infraestructuras capaces de manejar hasta un millón de tokens de manera económica y precisa, mediante proveedores como OpenRouter. Modelos fundacionales como *DeepSeek V4 Pro* o variantes optimizadas de *Claude Opus* serán utilizados para este razonamiento avanzado. El desarrollador instruirá a su terminal local mediante las exportaciones pertinentes de variables de entorno, instruyendo a Claude Code a delegar la computación al enrutador externo, garantizando un límite superior de contexto vastamente expandido que retrase la compactación inicial de la sesión.

Intervención 2: Creación de la Memoria Inmutable (CLAUDE.md)

Para proporcionar estabilidad sistémica, el agente será instruido para escribir, antes de generar cualquier línea de código de Rust, un archivo CLAUDE.md en la raíz del proyecto. Este manifiesto actuará como la base de datos de referencias estáticas durante todo el ciclo de vida de desarrollo. Contendrá un diccionario jerárquico preciso de las traducciones léxicas (ej., `fk -> fn`, `estu -> let`, `ent32 -> i32`) y las especificaciones arquitectónicas de transpilación (el mandato ineludible de utilizar `pest` para la gramática y `quote!` para la emisión de Rust). Cada vez que la herramienta de terminal inicie, o después de ciertos resets de entorno, este archivo se inyectará nuevamente en el contexto del modelo de lenguaje, recordando de manera forzada las reglas operativas del compilador en Esperanto.

Intervención 3: Programación de Resiliencia mediante Hooks

La táctica más sofisticada de este esfuerzo de ingeniería reside en la manipulación del archivo de configuración profunda de Claude Code (`.claude/settings.json`), ubicado a nivel local del proyecto. Claude Code posee un motor de "Hooks", el cual permite interceptar eventos del ciclo de vida del agente y forzar la ejecución de rutinas de línea de comandos predefinidas, operando como salvaguardas que previenen degradaciones de la base de código incluso cuando el asistente olvida pasos vitales.

El desarrollador ordenará la inserción de las siguientes rutinas de persistencia de configuración en el objeto JSON de Hooks:

1. **Recuperación Post-Compactación:** Para contrarrestar la pérdida de detalles técnicos tras las condensaciones periódicas del LLM, se aprovechará una interacción indirecta del hook `SessionStart` con el discriminador `compact`. Esta rutina emitirá el contenido completo de las reglas léxicas directamente al flujo de entrada estándar (`stdin`) del agente en el milisegundo exacto posterior a la finalización de la compactación de contexto. La instrucción obliga al agente a priorizar el contexto del Esperanto reconstruido.
2. **Validación Continua (PostToolUse):** Un segundo gancho se enganchará al evento `PostToolUse` bajo el filtro de expresiones regulares de edición y escritura de archivos (`Edit|Write`). Tras cada modificación del analizador léxico o sintáctico originada por el modelo, el sistema forzará de manera transparente la ejecución del comando `cargo fmt` y

cargo check. Esta red de seguridad informa de inmediato al agente si el código generado introduce ambigüedades en la gramática PEG o rompe las variantes de enumeración de la biblioteca syn, precipitando una corrección inmediata antes de que la deuda técnica se acumule en la memoria de la sesión y deteriore el progreso del compilador.

Intervención 4: El Bucle de Construcción Iterativa

Finalmente, equipado con el modelo de gran contexto y los protectores arquitectónicos en su lugar, el prompt se divide en órdenes progresivas impartidas a la interfaz de terminal:

- **Paso 1:** Se solicita la estructuración del esqueleto general utilizando la infraestructura modular de Cargo. El agente generará el espacio de trabajo organizando las cajas de dependencias para el intérprete CLI principal, la sub-biblioteca del analizador sintáctico y el módulo puente del transpilador, integrando syn, quote, proc-macro2 y pest en los archivos de configuración correspondientes.
- **Paso 2:** El asistente redactará las intrincadas reglas de la Gramática de Expresiones de Análisis (archivo .pest), mapeando todos los caracteres UTF-8 extendidos propios del alfabeto del Esperanto y sentando las bases lógicas para la prevalencia operativa de los literales sobre los identificadores.
- **Paso 3:** Se programarán las enumeraciones recursivas que conforman el Árbol Sintáctico. El agente diseñará estructuras en Rust meticulosamente que envuelvan las sentencias abstractas en Esperanto, albergando la información de ubicación original del flujo de texto para la emisión correcta de errores.
- **Paso 4:** Se procederá a implementar el módulo generador de código, ordenando a Claude Code descender recursivamente a través de las ramas del Árbol Sintáctico propietario. El agente empleará la función de cuasi-comillas de Rust para envolver las declaraciones exóticas y escupir cadenas sintácticas nativas sin incurrir en violaciones de tipos.
- **Paso 5:** Por último, se consolidará un envoltorio (wrapper) binario global que oculte por completo la presencia de Rust al usuario final. Este programa aceptará archivos terminados en una extensión personalizada, gestionará todo el proceso léxico y de transpilación hacia un directorio oculto y canalizará subrepticamente al compilador del sistema anfitrión, ofreciendo como resultado un binario de altísimo rendimiento ejecutado íntegramente a partir de especificaciones escritas de forma elegante, coherente y predecible en el idioma universal.

Este detallado y estricto enfoque estratégico garantiza no solo el éxito teórico de un lenguaje sin ataduras al idioma dominante de la industria, sino la realización práctica impecable a través de procesos autónomos blindados.

Fuentes citadas

1. Amharic Programming Language Design | PDF - Scribd, <https://www.scribd.com/document/802234715/6749348274> 2. Non-English-based programming languages - Wikipedia, https://en.wikipedia.org/wiki/Non-English-based_programming_languages 3. eo/README.md at master · objectionary/eo - GitHub, <https://github.com/objectionary/eo/blob/master/README.md> 4. Esperanto vocabulary - Wikipedia, https://en.wikipedia.org/wiki/Esperanto_vocabulary 5. Esperanto's Familiar Vocabulary: A Language You Already Half-Know!, <https://esperanto-chicago.org/esperantos-familiar-vocabulary-a-language-you-already-half-know>

/ 6. Esperanto-English Glossary, <https://esperanto-panorama.net/angla/vortaro.htm> 7. fidelisrafael/esperanto-analyzer: Morphological and syntactic analysis of Esperanto sentences - GitHub, <https://github.com/fidelisrafael/esperanto-analyzer> 8. A practical introduction to parsing, <https://jhwlr.io/intro-to-parsing/> 9. Can you apply the same lexer rules to all programming languages?, <https://softwareengineering.stackexchange.com/questions/224050/can-you-apply-the-same-lexer-rules-to-all-programming-languages> 10. A Step Back: Lexers and Parsers - OpenCS, <https://fitech101.aalto.fi/en/courses/modern-and-emerging-programming-languages/part-2/4-a-step-back-lexers-and-parsers> 11. Source-to-source compiler - Wikipedia, https://en.wikipedia.org/wiki/Source-to-source_compiler 12. Compiler vs Transpiler - Anujkumar Yadav, <https://anujf0510.medium.com/compiler-vs-transpiler-f226772c1551> 13. Transpiling vs Compiling : r/ProgrammingLanguages - Reddit, https://www.reddit.com/r/ProgrammingLanguages/comments/6rz3yd/transpiling_vs_compiling/ 14. Overview of the compiler - Rust Compiler Development Guide, <https://rustc-dev-guide.rust-lang.org/overview.html> 15. Lexing, Parsing, Code Analysis & Optimization ie. the Rust Compiler rustc Lexer HIR - GitHub Pages, <https://conf42.github.io/static/slides/Shriram%20Balaji%20-%20Conf42%20Rustlang%202024.pdf> 16. LLVM IR: The Esperanto of Computer Languages - Pat Shaughnessy, <https://patshaughnessy.net/2022/2/19/llvm-ir-the-esperanto-of-computer-languages> 17. Writing an LLVM backend for the Move language in Rust, <https://brson.github.io/2023/03/12/move-on-llvm/> 18. Building a Custom Compiler with LLVM: An Advanced Guide for Language Design - Medium, https://medium.com/@sohail_saifi/building-a-custom-compiler-with-llvm-an-advanced-guide-for-language-design-996fb9722751 19. Difference Between Transpiler and Compiler - GeeksforGeeks, <https://www.geeksforgeeks.org/compiler-design/difference-between-transpiler-and-compiler/> 20. immunant/c2rust: Migrate C code to Rust - GitHub, <https://github.com/immunant/c2rust> 21. C2Rust Manual, <https://c2rust.com/manual/print.html> 22. Hayroll: A Modular Wrapper for Translating C Macros and Conditional Compilation to Rust - University of Washington, <https://homes.cs.washington.edu/~mernst/pubs/c-rust-macros-pldi2026.pdf> 23. milahu/awesome-transpilers - GitHub, <https://github.com/milahu/awesome-transpilers> 24. Claude Code vs Cline (2026): Speed, Pricing, and Model Support Compared | Morph, <https://www.morphllm.com/comparisons/cline-vs-claude-code> 25. How Claude Code works - Claude Code Docs, <https://code.claude.com/docs/en/how-claude-code-works> 26. Claude Code - OpenRouter, <https://openrouter.ai/apps/claude-code> 27. KomputeKo - Wikipedia, <https://en.wikipedia.org/wiki/KomputeKo> 28. Plena Ilustrita Vortaro de Esperanto - Vikipedio, https://eo.wikipedia.org/wiki/Plena_Ilustrita_Vortaro_de_Esperanto 29. Angla-Esperanto Komputilia Vortaro - EVENTOJ, <http://www.eventoj.hu/steb/vortaroj/komputila-vortaro.htm> 30. KomputeKo, <https://komputeKo.net/> 31. Proper Esperanto words for modern technology / Alti temi / Forum - Lernu.net, <https://lernu.net/forumo/temo/663?hl=it> 32. Keywords - The Rust Reference, <https://doc.rust-lang.org/reference/keywords.html> 33. A - Keywords - The Rust Programming Language, <https://doc.rust-lang.org/book/appendix-01-keywords.html> 34. A - Keywords - The Rust Programming Language - MIT, https://web.mit.edu/rust-lang_v1.25/arch/amd64_ubuntu1404/share/doc/rust/html/book/second-edition/appendix-01-keywords.html 35. Types - The Rust Reference, <https://doc.rust-lang.org/reference/types.html> 36. Primitive Types - The Rust Programming Language, https://web.mit.edu/rust-lang_v1.25/arch/amd64_ubuntu1404/share/doc/rust/html/book/first-edition/

on/primitive-types.html 37. Primitives - Rust By Example - Rust Documentation, <https://doc.rust-lang.org/rust-by-example/primitives.html> 38. Crate std - Rust Documentation, <https://doc.rust-lang.org/std/> 39. JolonB/esperanto-python - GitHub, <https://github.com/JolonB/esperanto-python> 40. Introduction - The Rust Reference - Rust Documentation, <https://doc.rust-lang.org/reference/> 41. VERT: Verified Equivalent Rust Transpilation with Large Language Models as Few-Shot Learners - arXiv, <https://arxiv.org/html/2404.18852v2> 42. Creating a high level language that transpiles to rust, <https://langdev.stackexchange.com/questions/1611/creating-a-high-level-language-that-transpile-s-to-rust> 43. Documentation - Rust API Guidelines, <https://rust-lang.github.io/api-guidelines/documentation.html> 44. Translating Large-Scale C Repositories to Idiomatic Rust - arXiv, <https://arxiv.org/html/2511.20617v1> 45. Building a next generation Esperanto web dictionary - Wilfred Hughes, <https://www.wilfred.me.uk/blog/2011/01/22/building-a-next-generation-esperanto-web-dictionary/> 46. Lexing and parsing - Rust Compiler Development Guide, <https://rustc-dev-guide.rust-lang.org/the-parser.html> 47. Building a Compiler with Rust and LLVM | by Lyle Dean - loveholidays tech, <https://tech.loveholidays.com/building-a-compiler-with-rust-and-llvm-07068faf6acc> 48. Create Your Own Programming Language with Rust: Introduction, <https://createlang.rs/> 49. Writing a Compiler in Rust #1: Lexical Analysis - krypticismouse, <https://krypticismouse.hashnode.dev/writing-a-compiler-lexical-analysis> 50. How to Write a Programming Language: Part 1, The Lexer - ACCU, https://accu.org/journals/overload/26/145/balaam_2510/ 51. syn - kernel - Rust, <https://rust.docs.kernel.org/syn/index.html> 52. syn - crates.io: Rust Package Registry, <https://crates.io/crates/syn> 53. syn - Rust - Docs.rs, <https://docs.rs/syn> 54. syn - Rust - Google, <https://google.github.io/zerocopy/syn/index.html> 55. parse_quote in syn - Rust - Docs.rs, https://docs.rs/syn/latest/syn/macro.parse_quote.html 56. Context Compression in AI Agents: Hermes vs. Claude Code - Mem0, [https://mem0.ai/blog/how-hermes-and-claude-handle-context-compression-in-real-production-agents-\(and-what-you-should-extract\)](https://mem0.ai/blog/how-hermes-and-claude-handle-context-compression-in-real-production-agents-(and-what-you-should-extract)) 57. Claude Code Context Backups: Beat Auto-Compaction, <https://claudefa.st/blog/tools/hooks/context-recovery-hook> 58. Claude Code: Post-Compaction Hooks for Context Renewal | by Nick Porter | Medium, <https://medium.com/@porter.nicholas/claude-code-post-compaction-hooks-for-context-renewal-7b616dcaa204> 59. Own Your Harness. What compaction in Claude Code and... | by Sau Sheong | Apr, 2026, <https://sausheong.com/own-your-harness-2f5299a855a7> 60. 1M context in Claude Code — is it actually 1M or just a router with a summary handoff at 200K? : r/ClaudeCode - Reddit, https://www.reddit.com/r/ClaudeCode/comments/1rvz52c/1m_context_in_claude_code_is_it_actually_1m_or/ 61. Pseudo-PostCompact Hook—Reminding Claude of what it should already know - Reddit, https://www.reddit.com/r/ClaudeAI/comments/1qws098/pseudopostcompact_hookreminding_claude_of_what_it/ 62. How to Use OpenRouter with Claude Code: Run Cheaper Models as a Backend, <https://www.mindstudio.ai/blog/how-to-use-openrouter-with-claude-code-cheaper-models> 63. How to Run Claude Code with Cheaper Models: OpenRouter, NVIDIA NIM, and Ollama, <https://www.mindstudio.ai/blog/claude-code-cheaper-models-openrouter-nvidia-nim-ollama> 64. DeepSeek-V4-Pro - Nvidia NIM, <https://build.nvidia.com/deepseek-ai/deepseek-v4-pro/modelcard> 65. DeepSeek-V4-Pro - DeepInfra, <https://deepinfra.com/deepseek-ai/DeepSeek-V4-Pro> 66. DeepSeek-V4-Pro

download | SourceForge.net, <https://sourceforge.net/projects/deepseek-v4/> 67. DeepSeek V4 Pro - API Pricing & Benchmarks - OpenRouter, <https://openrouter.ai/deepseek/deepseek-v4-pro>

68. Integrate with Claude Code | DeepSeek API Docs, https://api-docs.deepseek.com/quick_start/agent_integrations/claude_code 69. Automate actions with hooks - Claude Code Docs, <https://code.claude.com/docs/en/hooks-guide> 70. Claude Code settings - Claude Code Docs, <https://code.claude.com/docs/en/settings> 71. Claude Code settings.json: Complete config guide (2026) - eesel AI, <https://www.eesel.ai/blog/settings-json-claude-code> 72. Where Is Claude Code settings.json? 5 Config Files, 1 Priority Rule - Vincent, <https://blog.vincentqiao.com/en/posts/claude-code-settings-intro/> 73. Hooks reference - Claude Code Docs, <https://code.claude.com/docs/en/hooks> 74. Understanding Claude Code hooks documentation - PromptLayer Blog, <https://blog.promptlayer.com/understanding-claude-code-hooks-documentation/> 75. PreCompact and PostCompact Hooks - Claude Code - Developers Digest, <https://www.developersdigest.tech/guides/pre-post-compact-hook> 76. Claude Code & Agent SDK Hooks (2026): PreToolUse, PostToolUse, UserPromptSubmit, Stop, SubagentStop - Morph, <https://www.morphllm.com/claude-code-hooks> 77. Claude Code Hooks: Automate Your AI Coding Workflow - Kyle Redelinghuys, <https://www.ksred.com/claude-code-hooks-a-complete-guide-to-automating-your-ai-coding-workflow/> 78. Getting Started - The Rust Programming Language - MIT, https://web.mit.edu/rust-lang_v1.25/arch/amd64_ubuntu1404/share/doc/rust/html/book/first-edition/getting-started.html